

Core Language Working Group "tentatively ready" Issues for the November, 2017 (Albuquerque) meeting

Section references in this document reflect the section numbering of document [WG21 N4700](#).

1862. Determining “corresponding members” for friendship

Section: 17.6.4 [temp.friend] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2014-02-13

During the discussion of issue 1804, it was noted that the process of determining whether a member of an explicit or partial specialization corresponds to a member of the primary template is not well specified. In particular, it should be clarified that the primary template should not be instantiated during this process; instead, the template arguments from the specialization should simply be substituted into the member declaration.

Proposed resolution (October, 2017):

Change 17.6.4 [temp.friend] paragraph 4 as follows:

A **template friend declaration may declare a member of a class template may be declared dependent type** to be a friend **of a non-template class**. The friend declaration shall declare a function or specify a type with an *elaborated-type-specifier*, in either case with a *nested-name-specifier* ending with a *simple-template-id*, *C*, whose *template-name* names a class template. The template parameters of the template friend declaration shall be deducible from *C* (17.9.2.5 [temp.deduct.type]). In this case, the corresponding member of every specialization of the primary class template and class template partial specializations thereof a member of a specialization *S* of the class template is a friend of the class granting friendship. For explicit specializations and specializations of partial specializations, the corresponding member is the member (if any) that has the same name, kind (type, function, class template, or function template), template parameters, and signature as the member of the class template instantiation that would otherwise have been generated if deduction of the template parameters of *C* from *S* succeeds, and substituting the deduced template arguments into the friend declaration produces a declaration that would be a valid redeclaration of the member of the specialization. [Example:

```
template<class T> struct A {
    struct B { };
    void f();
    struct D {
        void g();
    };
    T h();
    template<T U> T i();
};
template<> struct A<int> {
    struct B { };
    int f();
    struct D {
        void g();
    };
    template<int U> int i();
};
template<> struct A<float*> {
    int *h();
};
class C {
    template<class T> friend struct A<T>::B;           // grants friendship to A<int>::B even though
                                                    // it is not a specialization of A<T>::B
    template<class T> friend void A<T>::f();           // does not grant friendship to A<int>::f()
                                                    // because its return type does not match
    template<class T> friend void A<T>::D::g();         // does not grant friendship to A<int>::D::g()
                                                    // because A<int>::D is not a specialization of A<T>::D ill-formed, A<T>::D does not end with a simple
    template<class T> friend int *A<T>::h();           // grants friendship to A<int*>::h() and A<float*>::h()
    template<class T> template<T U>
        friend T A<T>::i();                           // grants friendship to instantiations of A<T>::i() and to A<int>::i()
                                                    // and thereby to all specializations of those function templates
};
```

—end example]

2177. Placement operator delete and parameter copies

Section: 8.3.4 [expr.new] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2015-09-30

Consider the following example:

```
void *operator new(size_t n, std::string s) {
    std::string t = std::move(s);
    std::cout << "new " << t << std::endl;
```

```

    return operator new(n);
}
void operator delete(void*, std::string s) {
    std::cout << "delete " << s << std::endl;
}
struct X { X() { throw 0; } };
int main() {
    try {
        new ("longer than the small string buffer") X();
    } catch (...) {}
}

```

Current implementations print

```

new longer than the small string buffer
delete

```

because the same `std::string` object is used for both the new and delete calls. We should introduce additional copies to separate out the parameters in this case or make non-trivially-copyable parameter types ill-formed here.

Notes from the October, 2015 meeting:

CWG favored limiting the parameters of an overloaded deallocation function to trivially-copyable types.

Proposed resolution (October, 2017):

Change 8.3.4 [expr.new] paragraph 24 as follows:

If a *new-expression* calls a deallocation function, it passes the value returned from the allocation function call as the first argument of type `void*`. If a placement deallocation function is called, it is passed the same additional arguments as were passed to the placement allocation function, that is, the same arguments as those specified with the *new-placement* syntax. If the implementation is allowed to **introduce a temporary object or** make a copy of any argument as part of the call to the allocation function, it is **allowed to make a copy (of the same original value) as part of the call to the deallocation function or to reuse the copy made as part of the call to the allocation function. If the copy is elided in one place, it need not be elided in the other unspecified whether the same object is used in the call to both the allocation and deallocation functions.**

2305. Explicit instantiation of constexpr or inline variable template

Section: 17.8.2 [temp.explicit] **Status:** tentatively ready **Submitter:** John Spicer **Date:** 2016-07-14

According to 17.8.2 [temp.explicit] paragraph 1,

An explicit instantiation of a function template or member function of a class template shall not use the `inline` or `constexpr` specifiers.

Should this apply to explicit specializations of variable templates as well?

(See also issues 1704 and 1728).

Proposed resolution (August, 2017):

Change 17.8.2 [temp.explicit] paragraph 1 as follows:

A class, function, variable, or member template specialization can be explicitly instantiated from its template. A member function, member class or static data member of a class template can be explicitly instantiated from the member definition associated with its class template. An explicit instantiation of a function template ~~or~~, member function of a class template, **or variable template** shall not use the `inline` or `constexpr` specifiers.

2307. Unclear definition of “equivalent to a nontype template parameter”

Section: 17.7.2.1 [temp.dep.type] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2016-07-21

The description of whether a template argument is equivalent to a template parameter in 17.7.2.1 [temp.dep.type] paragraph 3 is unclear as it applies to non-type template parameters:

A template argument that is equivalent to a template parameter (i.e., has the same constant value or the same type as the template parameter) can be used in place of that template parameter in a reference to the current instantiation. In the case of a non-type template argument, the argument must have been given the value of the template parameter and not an expression in which the template parameter appears as a subexpression.

For example:

```

template<int N> struct A {
    typedef int T[N];
    static const int AlsoN = N;
    A<AlsoN>::T s; // #0, clearly supposed to be OK
    static const char K = N;
    A<K>::T t;     // #1, OK?
    static const long L = N;
    A<L>::T u;     // #2, OK?
    A<(N)>::T v;   // #3, OK?
    static const int M = (N);
}

```

```

    A<M>::T w;    // #4, OK?
};

```

#1 narrows the template argument. This obviously should not be the injected-class-name, because `A<257>::T` may well be `int[1]` not `int[257]`. However, the wording above seems to treat it as the injected-class-name.

#2 is questionable: there is potentially a narrowing conversion here, but it doesn't actually narrow any values for the original template parameter.

#3 is hard to decipher. On the one hand, this is an expression involving the template parameter. On the other hand, a parenthesized expression is specified as being equivalent to its contained expression.

#4 should presumably go the same way that #3 does.

Proposed resolution (August, 2017):

Change 17.7.2.1 [temp.dep.type] paragraph 3 as follows:

A template argument that is equivalent to a template parameter (i.e., has the same constant value or the same type as the template parameter) can be used in place of that template parameter in a reference to the current instantiation. **For a template type-parameter, a template argument is equivalent to a template parameter if it denotes the same type. For a non-type template parameter, a template argument is equivalent to a template parameter if it is an identifier that names a variable that is equivalent to the template parameter. A variable is equivalent to a template parameter if**

- it has the same type as the template parameter (ignoring cv-qualification) and**
- its initializer consists of a single identifier that names the template parameter or, recursively, such a variable.**

[Note: Using a parenthesized variable name breaks the equivalence. —end note] In the case of a non-type template argument, the argument must have been given the value of the template parameter and not an expression in which the template parameter appears as a subexpression. [Example:

```

template <class T> class A {
    A* p1;                // A is the current instantiation
    A<T>* p2;              // A<T> is the current instantiation
    A<T*> p3;              // A<T*> is not the current instantiation
    ::A<T>* p4;            // ::A<T> is the current instantiation
    class B {
        B* p1;            // B is the current instantiation
        A<T>::B* p2;       // A<T>::B is the current instantiation
        typename A<T*>::B* p3; // A<T*>::B is not the current instantiation
    };
};

template <class T> class A<T*> {
    A<T*>* p1;             // A<T*> is the current instantiation
    A<T>* p2;              // A<T> is not the current instantiation
};

template <class T1, class T2, int I> struct B {
    B<T1, T2, I>* b1;       // refers to the current instantiation
    B<T2, T1, I>* b2;       // not the current instantiation
    typedef T1 my_T1;
    static const int my_I = I;
    static const int my_I2 = I+0;
    static const int my_I3 = my_I;
    static const long my_I4 = I;
    static const int my_I5 = (I);
    B<my_T1, T2, my_I>* b3;  // refers to the current instantiation
    B<my_T1, T2, my_I2>* b4; // not the current instantiation
    B<my_T1, T2, my_I3>* b5; // refers to the current instantiation
    B<my_T1, T2, my_I4>* b6; // not the current instantiation
    B<my_T1, T2, my_I5>* b7; // not the current instantiation
};

```

—end example]

Additional note (November, 2017):

It was observed that the proposed resolution does not address partial specializations, which also depend on the definition of equivalence. For example:

```

template <typename T, unsigned N> struct A;
template <typename T> struct A<T, 42u> {
    typedef int Ty;
    static const unsigned num = 42u;
    static_assert(!A<T, num>::Ty(), "");
};
A<int, 42u> a; // GCC, MSVC, ICC accepts; Clang rejects

```

The issue is being returned to "review" status in order to consider these additional questions.

Notes from the November, 2017 (Albuquerque) meeting:

CWG decided to proceed with this resolution and deal with partial specialization in a separate issue.

Section: 11.5 [dcl.struct.bind] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2016-08-12

According to the current rules for structured binding declarations, the user-defined case declares the bindings as variables of reference type. This presumably makes an example like the following valid:

```
auto [a] = std::tuple<int>(0);
extern int &&a; // ok, redeclaration, could even be in a different TU
```

This seems unreasonable, especially in light of the fact that it only works for the user-defined case and not the built-in case (where the bindings are not modeled as references).

Proposed resolution (August, 2017):

Change 11.5 [dcl.struct.bind] paragraph 3 as follows:

Otherwise, if the *qualified-id* `std::tuple_size<E>` names a complete type, the expression `std::tuple_size<E>::value` shall be a well-formed integral constant expression and the number of elements in the *identifier-list* shall be equal to the value of that expression. The *unqualified-id* `get` is looked up in the scope of `E` by class member access lookup (6.4.5 [basic.lookup.classref]), and if that finds at least one declaration, the initializer is `e.get<i>()`. Otherwise, the initializer is `get<i>(e)` where `get` is looked up in the associated namespaces (6.4.2 [basic.lookup.argdep]). In either case, `get<i>` is interpreted as a *template-id*. [Note: Ordinary unqualified lookup (6.4.1 [basic.lookup.unqual]) is not performed. —end note] In either case, `e` is an lvalue if the type of the entity `e` is an lvalue reference and an xvalue otherwise. Given the type T_i designated by `std::tuple_element<i, E>::type`, each v_i is a variable **variables are introduced with unique names r_i** of type “reference to T_i ” initialized with the initializer (11.6.3 [dcl.init.ref]), where the reference is an lvalue reference if the initializer is an lvalue and an rvalue reference otherwise. **Each v_i is the name of an lvalue of type T_i that refers to the object bound to r_i** ; the referenced type is T_i .

2315. What is the “corresponding special member” of a variant member?

Section: 15.8 [class.copy] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2016-08-15

According to 15.8.1 [class.copy.ctor] bullet 10.1,

A defaulted copy/move constructor for a class `x` is defined as deleted (11.4.3 [dcl.fct.def.delete]) if `x` has:

- a variant member with a non-trivial corresponding constructor and `x` is a union-like class,

However, it is not clear from this specification how to handle an example like:

```
struct A {
    A();
    A(const A&);
};
union B {
    A a;
};
```

since there is no corresponding special member in `A`.

Proposed resolution (August, 2017):

Change 15.8.1 [class.copy.ctor] paragraph 10 as follows:

An implicitly-declared copy/move constructor is an inline public member of its class. A defaulted copy/move constructor for a class `x` is defined as deleted (11.4.3 [dcl.fct.def.delete]) if `x` has:

- ~~a variant member with a non-trivial corresponding constructor and `x` is a union-like class,~~
- a potentially constructed subobject type `M` (or array thereof) that cannot be copied/moved because overload resolution (16.3 [over.match]), as applied to find `M`'s corresponding constructor, results in an ambiguity or a function that is deleted or inaccessible from the defaulted constructor,
- **a variant member whose corresponding constructor as selected by overload resolution is non-trivial,**
- any potentially constructed subobject of a type with a destructor that is deleted or inaccessible from the defaulted constructor, or,
- for the copy constructor, a non-static data member of rvalue reference type.

2338. Undefined behavior converting to short enums with fixed underlying types

Section: 8.2.9 [expr.static.cast] **Status:** tentatively ready **Submitter:** Thomas Köppe **Date:** 2017-03-24

The specifications of `std::byte` (21.2.5 [support.types.byteops]) and `bitmask` (20.4.2.1.4 [bitmask.types]) have revealed a problem with the integral conversion rules, according to which both those specifications have, in the general case. undefined behavior. The problem is that a conversion to an enumeration type has undefined behavior unless the value to be converted is in the range of the enumeration.

For enumerations with an unsigned fixed underlying type, this requirement is overly restrictive, since converting a large value to an unsigned integer type is well-defined.

Proposed resolution (August, 2017):

Change 8.2.9 [expr.static.cast] paragraph 10 as follows:

A value of integral or enumeration type can be explicitly converted to a complete enumeration type. ~~The~~ **If the enumeration type has a fixed underlying type, the value is first converted to that type by integral conversion, if necessary, and then to the enumeration type. If the enumeration type does not have a fixed underlying type, the** value is unchanged if the original value is within the range of the enumeration values (10.2 [dcl.enum]). ~~Otherwise, and otherwise,~~ the behavior is undefined.